Audit 11

Technical Debt

TradeVault — Pre-Seed Technical Due Diligence

Lead Engineer

Generated: July 2026 | 7 modules | 206 source files

1. Executive Summary

This audit evaluates the technical debt accumulated across the TradeVault codebase. Score: 45/100 — significant debt that will slow feature delivery and increase bug rates.

Overall Technical Debt Score	45/100 (45%)
Dead / Obsolete Code	38/100 (38%)
Dependency Health	55/100 (55%)
Architectural Compliance	42/100 (42%)
Code Quality Consistency	50/100 (50%)
Documentation / TODOs	40/100 (40%)

2. Dead & Obsolete Code

[P0] AI Platform: ~40% scaffolding with zero runtime usage

The AI module (src/modules/ai/) contains 22 files including agent registry, router, MCP types, RAG types, job types, telemetry, context builder, prompt builder, response formatter, and tool runtime. Only 4 of these are actually imported from app/:
- src/modules/ai/memory.ts (loadMemory, remember)
- src/modules/ai/context.ts (AIUserContext type)
- src/modules/ai/fallback-coach.ts (fallbackCoachAnswer)
- src/modules/ai/provider-service.ts (generate, runWithTools)

Files with zero imports from app/:
- agents/catalog.ts, agents/registry.ts, agents/types.ts, agents/coach.agent.ts
- context-builder.ts, prompt-builder.ts, response-formatter.ts
- infra.ts (85-line re-export barrel with unused exports)
- jobs/types.ts, mcp/types.ts, rag/types.ts
- router/router.ts, router/types.ts
- telemetry.ts, tools/runtime.ts, tools/types.ts

Estimated: 1,200+ lines of scaffolding that will never run without major rewrites.

[P1] AI Provider module has unused providers

src/modules/ai-provider/ contains 6 files. Only resolveProvider() and resolveToolCapableProvider() from registry.ts are used. The Anthropic, Gemini, and OpenAI provider implementations all exist but the app only ever calls generate() through the provider-service abstraction. The individual provider files add ~180 lines of dead weight.

[P1] Voice module: half-scaffolded, likely unused

src/modules/voice/ has 5 files (index, localVoice, profile, prosody, types). The jarvisVoice utility imports { JARVIS_VOICE, pickJarvisVoice, toSpeechSegments }, but there's no evidence the voice synthesis feature is actually wired into the UI. The prosody.ts stripForSpeech function is exported but never imported elsewhere.

[P2] Empty re-export barrels

src/modules/trading/analysis/index.ts and src/modules/automation/index.ts are minimal re-export stubs that barely add value. They exist for 'future extensibility' but currently serve only as indirection.

[P2] Duplicate cn() utility across layers

src/app/utils/cn.ts exists only to re-export from src/shared/ui/cn.ts. While the comment explains this is intentional, it adds a maintenance surface that could break if the re-export path changes. Prefer direct imports.

3. Dependency Analysis

20 production dependencies, 17 dev dependencies

[P2] cmdk (Command menu) — likely unused

The cmdk package (~8KB) is listed as a dependency but there is no command palette visible in the UI. It may be intended for future use or for the AI assistant, but currently adds bundle weight with zero user-facing value.

[P2] remark-gfm and react-markdown — single-use overhead

Both are imported only in the AiAssistant component for rendering markdown responses. This is valid but heavy for a single component. If AI responses are mostly simple text, a lighter parser could suffice.

[P2] tw-animate-css — animation library debt

Adding a CSS animation library before core animations are finalized risks needing to replace it later. Ensure this is truly needed vs vanilla CSS transitions.

[P1] No TypeScript path aliases for tests

Tests use relative imports (../../src/modules/...) instead of @/ aliases. This makes refactoring harder and imports brittle across the 10 test files.

[P2] nitro (devDependency) — build-only tool

Nitro is in devDependencies but there's no evidence it's used in the build pipeline (the build script uses vite build). This may be a leftover from an earlier version.

[P3] All deps are on current-gen versions — good

No severely outdated dependencies found. The stack (TanStack Start, Vite 7, React 19) is modern. Bun lockfile is clean at 228KB.

4. Architectural Debt

[P0] Layer dependency violation: modules import from app/ (8 sites)

The architecture rule states: 'src/tradevault/ (UI) imports src/modules/ — never the inverse.' 8 violations found in modules/:
- modules/ai/memory.ts: imports generateId from @/app/store
- modules/discipline/engine.ts: imports Trade from @/app/types, checkTradeAgainstRules from @/app/utils/tradingRules
- modules/discipline/types.ts: imports TradingRule from @/app/utils/tradingRules
- modules/automation/types.ts, events/types.ts, trading/analysis/engine.ts: import Trade from @/app/types
- modules/notifications/engine.ts: imports generateId from @/app/store

Consequence: modules cannot be extracted, tested in isolation, or reused.

[P0] Event Bus: fire-and-forget with no error handling

The EventBus implementation (src/modules/events/bus.ts) uses a simple listener map with no async handling, no error boundaries, no retry, no dead-letter queue. Handlers that throw silently crash the event chain. This was flagged in Audit 01 as P0 and remains unfixed.

[P1] Monolithic pages (Checklist.tsx = 2030 lines)

The Checklist page is 2,030 lines of mixed concerns: React components, event handlers, business logic, inline styles, and i18n lookups. It should be split into containers, presentation, and business logic modules. Same applies to Landing.tsx (1412 lines) and Analytics.tsx (1102 lines).

[P1] Store module spans product/store/cache concerns

src/app/store.ts exports generateId alongside localStorage persistence and state management. generateId is a pure utility that should live in a shared utils file. Modules importing @/app/store for generateId pull in unrelated storage concerns.

[P1] No formal error boundary hierarchy

Error handling is ad-hoc with try/catch scattered across files. There's no React Error Boundary component, no error boundary tree, and no consistent error recovery strategy. Errors can leave UI in inconsistent states.

[P2] Inline styles instead of design tokens (15+ files)

15+ components use style={{}} instead of Tailwind utility classes or design tokens. This bypasses the design system, makes theming harder, and creates visual inconsistency. Files: AccountSwitcher.tsx (11 inline styles), Landing.tsx (9), ThemeSettings.tsx (7), etc.

[P2] console.log scattered across 20+ files

console.log/error/warn used in 20+ source files. Only server.ts and shared/error-reporting.ts have structured logging. This will become unmanageable in production and creates noise.

5. TODO / FIXME / Suppression Density

[P2] Surprisingly few TODO/FIXME/HACK markers (only 2)

Only src/app/i18n/locales/pt.ts and es.ts have 1 TODO each. Either the team doesn't leave markers, or they were stripped. Low marker count does NOT mean low debt — the architectural debt above was found during audit without marker guidance.

[P1] Heavy TypeScript suppression usage (14 files)

14 files use @ts-expect-error, @ts-ignore, or eslint-disable. Top offenders:
- goalPlan.ts: 3 suppressions
- tradingPlan.ts: 2
- Checklist.tsx: 2
- CalendarPage.tsx: 2
- usePushNotifications.ts: 2

These suppress real type issues rather than fixing them, creating latent bugs.

[P1] Heavy 'as any' usage (12+ files)

12+ files use 'as any' or 'as unknown' type casts. routeTree.gen.ts has 4, goalPlan.ts has 4, usePushNotifications.ts has 4. Each cast bypasses TypeScript safety guarantees.

[P1] try/catch without typed error handling

While many functions have try/catch, none use a custom Error class or typed error handling. All errors are caught as generic and re-thrown or logged as unknown. This makes error diagnosis and monitoring harder than necessary.

6. Localization Debt

[P1] Severe translation imbalance: French has 3x more keys

src/app/i18n/locales/fr.ts has 1,023 translation keys across 1,069 lines.
All other 10 locales have ~297-305 keys each (~300 lines).

This means 70% of the UI strings are only available in French and English. The 10 non-French locales are missing ~700 keys each, resulting in a partial UX for international users. This is ~7,000 missing translations total.

This strongly suggests the product was French-first and i18n was added later without completing the translations.

[P2] Translation file structure lacks type safety

Translations are plain .ts objects without a shared type definition. Adding or removing a key in one locale won't cause type errors in others. A shared TranslationKeys type or codegen step would prevent drift.

7. Testing Debt

[P1] 10 test files for 206 source files (4.8% coverage)

Only 10 test files exist in tests/, totalling ~995 lines. No UI/component tests, no integration tests, no E2E tests exist. The 10 tests focus on:
- Trade calculations (149 lines)
- AI infrastructure (169 lines)
- Coaching logic (129 lines combined)
- Statistical edge analysis (195 lines)

Zero tests for: pages (20+), components (15+), backend (10+), hooks (5+), automation, notifications, discipline engine, event bus, billing, push crypto.

[P2] No test runner configured in package.json

package.json has no test script. While Vitest is likely available via Vite, there is no standardized way to run tests. The tests/ directory exists but team members must manually invoke the test runner.

[P2] Tests import source directly without path aliases

Tests use relative imports like '../src/modules/ai/infra'. If source files are moved, tests will silently break. @/ aliases are configured in tsconfig but not used in any test file.

8. Documentation & Configuration Debt

[P2] docs/ directory is comprehensive but partially stale

11 documentation files exist (architecture.md, ai-architecture.md, product.md, roadmap.md, etc.) but some refer to patterns that no longer match the codebase. docs/features-status.md would need updating after this audit series.

[P3] No CONTRIBUTING.md or CODE_OF_CONDUCT.md

For a pre-seed project this is acceptable, but before open-sourcing or hiring, a CONTRIBUTING.md with setup instructions, coding standards, and PR workflow will be needed. Currently only CLAUDE.md serves as team guidelines.

[P3] No .env.example up to date

The .env file contains actual Supabase keys. A .env.example with placeholder values should be committed to make onboarding reproducible.

9. Score Summary & Remediation Plan

Dead & Obsolete Code	38/100 (38%)
Dependency Health	55/100 (55%)
Architectural Compliance	42/100 (42%)
Code Quality Consistency	50/100 (50%)
TypeScript Hygiene	45/100 (45%)
Localization Completeness	35/100 (35%)
Test Coverage	18/100 (18%)
Documentation	60/100 (60%)

Priority Remediation (by effort)

- Week 1-2: Extract modules dependencies (8 violations) — unblocks all module testing
- Week 1-2: Add Event Bus error handling + dead-letter queue
- Week 3-4: Strip AI scaffolding (remove unused 18 files)
- Week 3-4: Fix TypeScript suppressions (14 files) and 'as any' casts
- Week 5-6: Split monolithic pages (Checklist, Landing, Analytics)
- Week 5-6: Complete i18n for all 10 locales (~7,000 keys)
- Week 7-8: Add Vitest config + test script + component test setup
- Week 7-8: Remove unused deps (cmdk, nitro) and stabilize animation lib
- Week 9-10: Add Error Boundary hierarchy and structured logging
- Ongoing: Replace inline styles with design tokens